Modeling Human Activity States Using Hidden Markov Models

Formative 2: Human Activity Recognition for inactivity and sudden-movement monitoring

Author: Eddy Irasetsa (individual assignment) | Repository: https://github.com/Eddydev-ALU/hmm-human-activity-recognition-formative-2

1. Background and Motivation

More and more older adults live alone, and two kinds of events put them at risk if nobody notices quickly: a long period of unusual inactivity (for example after a fall or a sudden illness) and a sudden, violent movement that may be a fall or a seizure. My use case is a phone or wearable monitor for older adults that infers the wearer's activity continuously from its built-in accelerometer and gyroscope, and alerts a caregiver when either kind of event appears. Human activity recognition (HAR) is needed because the raw sensor stream is noisy and the true activity is never observed directly. It has to be inferred from patterns in the signal and from how activities follow each other over time. A Hidden Markov Model (HMM) fits this naturally: the activity is a hidden state that persists and changes with learnable probabilities, and each short window of sensor data is a noisy observation of that state. I model four states: standing (upright, low-energy sway), walking (normal mobility), still (the phone is not being carried or the person is not moving at all, i.e. the inactivity alarm) and jumping (a high-energy, impact-rich movement that I use as a safe-to-record stand-in for sudden violent motion such as a fall or seizure).

2. Data Collection and Pre-processing

Table 1. *Devices and sampling rates (training phone and second test phone).*

Person	Phone	App / OS	Configured rate	Detected rate (from timestamps)
Eddy Irasetsa	iPhone XS	Sensor Logger 1.68 (iOS 18.7.5)	50 Hz (sampleRateMs = 20)	49.76–49.77 Hz → resampled to 50 Hz
A friend (shared test data only, not a group member)	Samsung SM-A235F	Sensor Logger 1.68 (Android)	100 Hz (sampleRateMs = 10)	102.07 Hz → low-pass + resampled to 50 Hz

Data were recorded with the Sensor Logger app (accelerometer and gyroscope, x/y/z, with timestamps), exported per recording as CSV files. For standing, walking and jumping the phone was carried at waist level; for still it lay on a flat table. I recorded 52 training files, 13 per activity, each about 13–16 s including about 2 s of button-tap lead-in and lead-out. After trimming, every file holds 8.7–12.5 s of clean activity. Every activity exceeds the required 90 s (Table 2). The iOS Accelerometer.csv contains user acceleration with gravity removed (m/s²), and the gyroscope gives angular velocity (rad/s). The Android export uses the same units, so no unit conversion was needed. To get new data for testing, a friend (not a group member; this assignment was done individually) shared one about 10 s recording per activity from their own phone (Table 1). These files were used only as unseen test data (Section 4).

Table 2. *Training data per activity.*

Activity	Files	Raw seconds	Clean seconds (after trim)	2 s windows
Standing	13	179.1	127.3	108
Walking	13	177.7	125.9	107
Jumping	13	176.9	125.1	106
Still	13	175.5	123.7	104
Total	52	709.2	502.0	425

Cleaning pipeline. (1) Labels are parsed from the leading letters of the folder name (handles both <label>_<NN>-<ts> and the second phone's <Label><NN>-<ts>). An assertion rejects unexpected labels. (2) Accelerometer and gyroscope are selected by column name (the export order is z, y, x) and merged on the nearest timestamp within half a sample. This is exact on the iPhone, where the sensors share timestamps, and robust on the Android phone, where 1–2 % of samples are offset. (3) Rows with missing values are dropped (none were found). (4) Every recording is resampled onto a uniform 50 Hz grid by linear interpolation. (5) 2 s are trimmed from each end to remove the phone taps. Raw files are never modified. The cleaned, labelled CSVs are saved in data/processed/.

Sampling-rate harmonisation. Sampling rates were checked rather than assumed. Each file's rate was detected from the median timestamp difference: 49.76–49.77 Hz on the iPhone (nominal 50 Hz) and 102.07 Hz on the Samsung (nominal 100 Hz). Every file is mapped onto the same exact 50 Hz grid, the rate of the training phone, by linear interpolation. For files sampled well above 50 Hz, a zero-phase 4th-order Butterworth low-pass filter at 20 Hz (0.8 × the new 25 Hz Nyquist frequency) is applied first, so the high-frequency content of the 100 Hz phone cannot alias into the 0–25 Hz band. Without harmonisation the same 100-sample window would last 1 s on one phone and 2 s on the other, and FFT features (dominant frequency, entropy) would not be comparable. On the iPhone, resampling also removes the 0.5 % clock drift.

Windowing logic. The window length is derived from the sampling rate: WIN = int(2.0 s × 50 Hz) = 100 samples, with 50 % overlap (50-sample hop, one observation per second). Walking (about 2 steps/s) and my jumping (about 1.5–2 jumps/s) therefore contribute at least two full cycles per window, which keeps the statistics stable. The FFT resolution of FS/N = 0.5 Hz is fine enough to separate the walking peak (about 1–2.5 Hz) from the jumping peak (about 3.5–4 Hz on the acceleration magnitude). Shorter windows hold under two cycles. Longer ones blur transitions and leave very few windows per 9.5 s file. Windows never span two recordings.

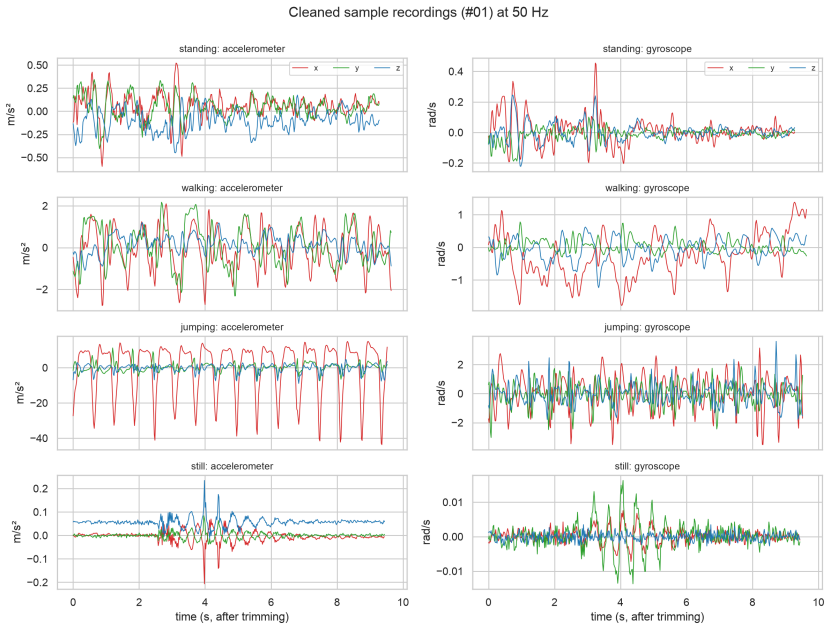


Figure 1. *Cleaned accelerometer (left) and gyroscope (right) signals, one recording per activity, at 50 Hz. Note the very different y-axis scales, from about ±0.1 m/s² (still) to about 40 m/s² (jumping).*

Video evidence (one 5 s video per activity, face and full body visible):

- **Standing:** https://drive.google.com/file/d/1f4_p_-ujCiJZu97DpZ5USixrpc7iwrEp/view?usp=sharing
- **Walking:** https://drive.google.com/file/d/1jgML2SbKMkt49ShfATg1yWFIp1IBQ9B9/view?usp=sharing
- **Jumping:** https://drive.google.com/file/d/1U_uXRKpqcJjFT1HDZSg_uy9OPgarakPx/view?usp=sharing
- **Still:** https://drive.google.com/file/d/1k5gLUf_np0WNJqVYsKyLhyXlgU_nX-zk/view?usp=sharing

3. HMM Setup and Implementation Details

Model components. Hidden states Z = {standing, walking, jumping, still}. Observations X are the Z-scored 17-dimensional feature vectors, one per window. Transition matrix A (4×4) with A_ij = P(z_t = j | z_t−1 = i). Initial distribution π. Emission model B: one multivariate Gaussian per state with diagonal covariance. A full 17×17 covariance would need 153 parameters per state from only about 100 windows and risks singular matrices, so I use the diagonal form. A variance floor of 0.01 (in standardised units) prevents a state from collapsing onto the almost-constant still features, which would make likelihoods explode.

Table 3. *Extracted features (12 time-domain, 5 frequency-domain) and their justification.*

Feature	Domain	Why it separates the activities
acc_rms, gyro_rms (log)	Time	Overall movement and rotation intensity: still ≪ standing < walking ≪ jumping (spans about 3 orders of magnitude).
acc_std_x/y/z, gyro_std_x/y/z (log)	Time	Per-axis variability: jumping is dominated by the vertical axis, walking spreads over all axes and gives hip rotation.
acc_sma (log)	Time	Signal magnitude area, a standard activity-energy measure that does not depend on orientation. It has the highest Fisher score.
acc_corr_xy/xz/yz	Time	Axis coupling: jumping moves the axes together (strong correlation), noise on a still phone is uncorrelated.
acc_dom_freq, gyro_dom_freq	Freq. (FFT)	Dominant frequency: walking about 1–2.5 Hz, jumping about 3.5–4 Hz (two impacts per jump), random for still.
acc_spec_energy (log)	Freq. (FFT)	Total spectral power excluding DC: how energetic the movement is.
acc/gyro_spec_entropy	Freq. (FFT)	Periodic motion gives a peaked spectrum (low entropy), sensor noise gives a flat one (high entropy). Separates still from walking and jumping.

Normalisation. Scale features are log10-transformed first, because they span several orders of magnitude and are strongly right-skewed. All features are then Z-score normalised, z = (x − μ_train)/σ_train. Z-scoring puts features with different units (m/s², rad/s, Hz, unit-less) on a common scale, so no feature dominates the Gaussian log-likelihood. It keeps the shape of each distribution (min-max scaling would be squashed by the jump spikes), and it keeps EM numerically well

conditioned. μ and σ are fitted on the training windows only and reused for the test data (saved in data/processed/scaler.json), so nothing leaks from the test set.

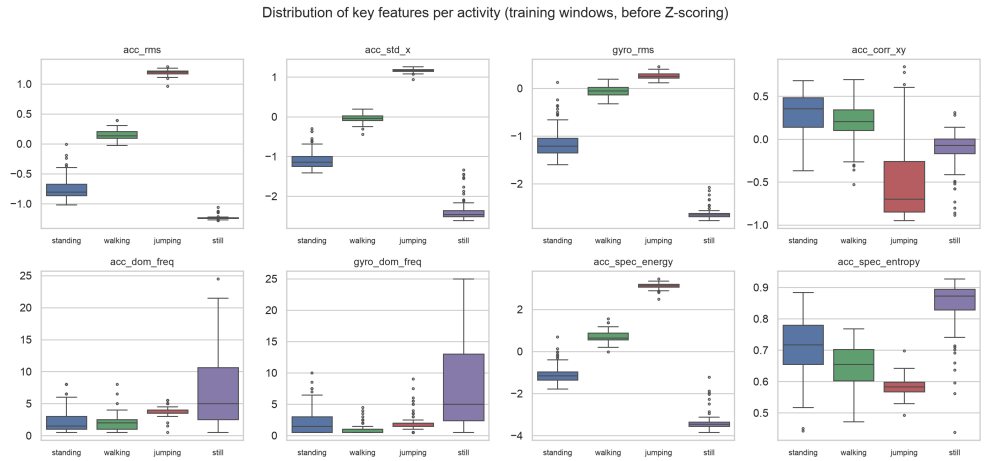


Figure 2. *Training-window distributions of key features per activity (before Z-scoring). The intensity features separate all four classes. Spectral entropy and dominant frequency separate periodic activities from noise.*

Algorithms and their link to the metrics. The HMM is written from scratch in numpy (GaussianHMM class; all recursions in log space with log-sum-exp to avoid underflow). The forward algorithm computes log P(X | λ) and serves as the training convergence criterion. Forward-backward provides the state posteriors for Baum-Welch (EM), which learns π, A and the Gaussian parameters over multiple sequences. Viterbi returns the single most likely state sequence, i.e. one hard label per window. These labels are exactly what accuracy, sensitivity, specificity and the confusion matrix are computed from, so Viterbi is the right decoder for my evaluation. Unit tests check that Viterbi and the forward likelihood match brute-force enumeration of all 3^6 paths on random small HMMs, that the rows of A and π sum to 1, and that the log-likelihood never decreases.

Training procedure. Each training file contains a single activity and therefore no transitions. To build a temporal context, I concatenated recordings into 13 training sequences. Each sequence holds the n-th recording of every activity in a random order (seed 42), with each recording's windows kept in time order. Emission means and variances were initialised from the labelled windows, and A and π from the label sequences with +1 Laplace smoothing (no zero probabilities). Baum-Welch then refined all parameters without using the labels. It stops when the total log-likelihood gain is below $\varepsilon = 10^{-4}$ (maximum 200 iterations). It converged after 6 iterations, from 675.1 to 826.4 (Figure 3), with a monotone increase. Hidden states were mapped to activity names with the Hungarian algorithm on the training confusion matrix. The mapping was the identity (0→standing, 1→walking, 2→jumping, 3→still), which shows that EM kept the meaning of each labelled initial state.

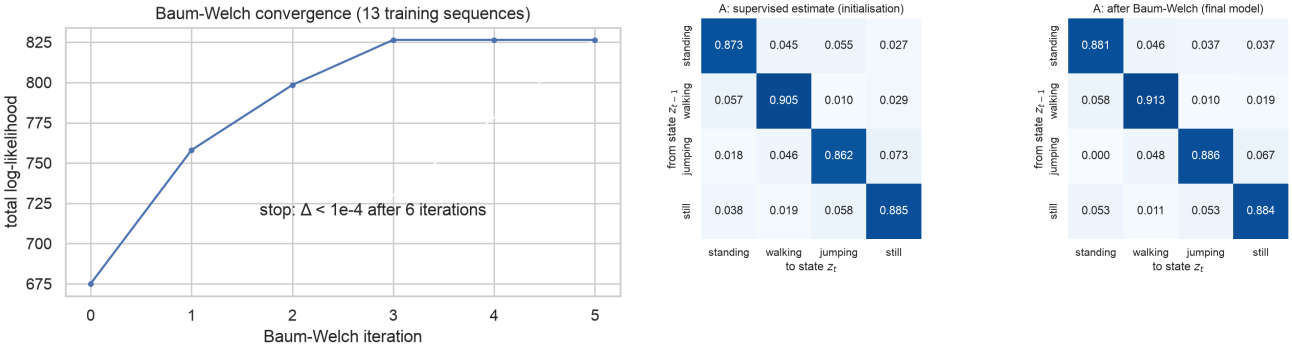


Figure 3. *Left: Baum-Welch total log-likelihood per iteration (stopping rule $\Delta < 10^{-4}$). Right: transition matrix A, supervised estimate vs after Baum-Welch.*

4. Results and Interpretation

Unseen test data. Two independent sources were used, and none of their files was used for training, normalisation or any design choice. (a) Same phone, new session: about 20 minutes after training (10:59–11:06 vs 09:32–10:40) I recorded three continuous recordings with all four activities back-to-back (standing→walking→jumping→still ×2, still→jumping→walking→standing ×1). These contain real transitions. Boundaries were labelled by hand from the motion-energy trace (the annotation file was empty), and the 8 windows covering the phone being put down or picked up were labelled 'transition' and excluded. (b) Different person, phone, rate and day: four single-activity files shared by a friend (not a group member) from the Samsung SM-A235F (100 Hz, 27–29 Oct). All files go through the identical pipeline with the training scaler and are Viterbi-decoded one recording at a time, giving 108 labelled test windows (88 + 20).

Table 4. *Evaluation on all unseen test data (both sources). Per-class values are one-vs-rest: sensitivity = TP/(TP+FN), specificity = TN/(TN+FP), accuracy = (TP+TN)/N. The last row gives the multi-class accuracy.*

State (Activity)	Number of Samples (windows)	Sensitivity	Specificity	Overall Accuracy
Standing	23	0.870	0.929	0.917
Walking	34	1.000	0.932	0.954
Jumping	32	0.938	1.000	0.982
Still	19	0.684	1.000	0.944
All (overall)	108	0.873 (macro)	0.966 (macro)	0.898

Table 5. *Unseen-data results per source.*

Unseen source	Windows	Accuracy	Macro sensitivity	Macro specificity	Still sensitivity
(a) same phone, new session (sequences)	88	0.875	0.833	0.957	0.571
(b) second phone and person (single-activity)	20	1.000	1.000	1.000	1.000

The model reaches 89.8% overall accuracy on all unseen data, against 98.4% on the training sequences. By source: 87.5% on the same-phone sequences and 100.0% on the second phone (20 of 20 windows). Walking is never missed (sensitivity 1.00) and jumping is almost perfect (sensitivity 0.94, specificity 1.00). The weak class is still: 13 of 19 still windows were recognised (sensitivity 0.68). All 6 misses were decoded as standing, and all of them are in the same-phone sequences. Still's specificity is 1.00, so whenever the model says still, it is right.

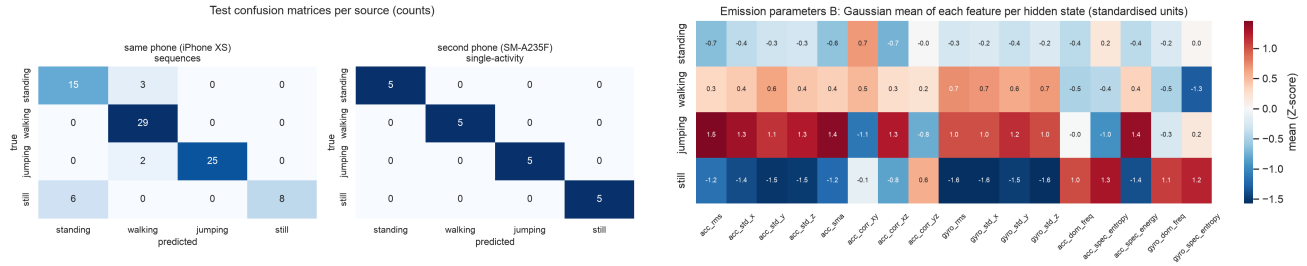


Figure 4. *Left: test confusion matrices (counts) for (a) the same-phone sequences and (b) the second phone. Right: emission parameters B, the per-state Gaussian means in Z-units.*

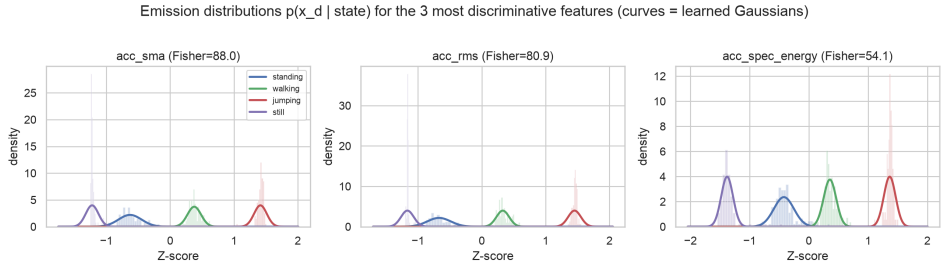


Figure 5. *Learned emission densities p(x_d | state) for the three most discriminative features (by Fisher score), over the training histograms.*

The emission plots show why most classes separate easily: on the intensity features the four Gaussians hardly overlap, in the order still < standing < walking < jumping. The narrow still cluster is widened by the variance floor, a deliberate trade-off between robustness and sharpness. The decoded sequences (Figure 6) show where the errors are: 10 of the 11 errors in the sequences lie within one window length (2 s) of an activity boundary, i.e. windows that partly cover two activities or the settling phone just after it is put down.

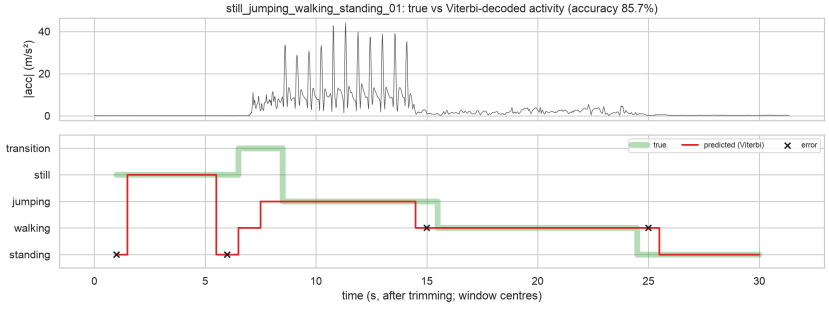


Figure 6. *True (green) vs Viterbi-decoded (red) activity for an unseen same-phone recording, with acceleration magnitude on top. Crosses mark errors. Decoded plots for all 7 test files are in the notebook and the figures/ folder.*

5. Discussion and Conclusion

Easiest and hardest activities. Jumping and walking were the easiest. Their energy (acc_sma, acc_rms) is one to two orders of magnitude above the static classes, and their spectra are periodic with distinct peaks. The only confusions were at their mutual boundary (2 jumping windows labelled walking where jumping ended). The hardest distinction was still vs standing. Both are 'no deliberate movement', and because iOS removes gravity from Accelerometer.csv, the model cannot use phone orientation (flat on a table vs upright at the waist), which would separate them trivially. It must rely on the small difference between the body sway of a standing person and pure sensor noise. In the same-phone sequences the missed still windows lie next to the moment the phone is put down or picked up (or right after the start tap), where residual vibrations look like standing sway. On the second phone, where the still file has no such gesture, all still windows were correct. Standing in turn was decoded as walking 3 times, each in a window straddling a walking boundary (twice at the first step of walking, once as walking ended). For the use case this bias is acceptable: missing the first seconds of inactivity only delays an inactivity alarm slightly, and the high-energy class that stands in for falls or seizures was detected from its first window. It was only missed in the last window of a bout, where jumping fades into walking.

Do the transition probabilities reflect realistic behaviour? Partly. The strong diagonal (self-transitions 0.88–0.91) is realistic: activities persist, and with a 1 s step it corresponds to expected bouts of about $1/(1-a_{ii}) \approx 8$–11 s. However, these values reflect the length of my 9–10 s recordings, not real daily life, where someone stays still or standing for minutes. The off-diagonal entries come from the random order in which I concatenated training recordings, so they encode no genuine behaviour. Baum-Welch did make them somewhat more plausible: for example jumping→still (0.067) is more likely than jumping→standing (0.000), which matches the order of my sessions. A realistic A would need long, naturally sequenced recordings (e.g. a day of wear).

Effect of sensor noise and sampling rate. Sensor noise matters most for the static classes. The still accelerometer noise is about 0.005 m/s^2, and the features of still windows are very tight, so without the variance floor the model would reject any still window with slightly higher noise. The floor trades some sharpness for robustness. The two phones differ in both rate (49.76 vs 102.07 Hz) and noise floor: the Samsung's still windows are quieter (mean log10 acc_rms -1.80 vs -1.24 on the iPhone). After anti-aliased resampling to 50 Hz these differences stayed much smaller than the gaps between classes, so the model transferred without retraining. 50 Hz (25 Hz Nyquist) is far above the 1–4 Hz movement frequencies, so the lower common rate cost nothing. However, the 2 s window, chosen for a 0.5 Hz FFT resolution, sets a 1–2 s time resolution, which is exactly the scale of the boundary errors above. The dominant-frequency features had low Fisher scores because for still and standing they are driven by noise.

Generalisation and limitations. The model generalises well. It is perfect on a different person, phone, sampling rate and day, even though that person's walking cadence differs (dominant frequency 1.5 vs 2.0 Hz). The drop from 98.4% (train) to 87.5% on the same-phone sequences comes almost entirely from boundaries and the settling phone, which never appear in single-activity training files. Two caveats: the second-phone set is small (20 windows) and has no transitions, so its 100 % is the easier case. In an ablation on all unseen data, the supervised-only HMM scored 93.5% and a frame-wise Gaussian classifier without transitions 89.8%, vs 89.8% for my final model. Baum-Welch slightly re-fitted the emissions at the cost of a few boundary windows, and the transition model added no accuracy because the classes are already well separated. I report this as observed and did not tune any choice on the test data.

Improvements. (1) More and more varied data: several participants, phone positions (pocket, hand, waist) and environments, and long recordings with natural transitions to learn a realistic A. (2) Additional sensors and features: the gravity vector or raw (uncalibrated) accelerometer for orientation, which would separate still from standing, plus jerk, band-energy ratios and zero-crossing rate. (3) Model variants: a full or tied covariance on decorrelated (PCA) features, Gaussian-mixture emissions, an explicit 'transition' state, or a hidden semi-Markov model with explicit duration distributions. (4) Shorter windows or a smaller hop for finer boundaries.

Conclusion. I collected 52 labelled recordings (more than 123 s per activity), harmonised them to 50 Hz, extracted 12 time-domain and 5 frequency-domain features, and built a Gaussian HMM from scratch with log-space Viterbi and multi-sequence Baum-Welch that converged in 6 iterations. On unseen data it reached 89.8% accuracy: 87.5% on new multi-activity sequences from my phone and 100.0% on a second person's 100 Hz Android phone, with near-perfect walking and jumping detection. Its main weakness is still vs standing near transitions. For an elderly-monitoring system, the next step is adding orientation information and long real-world recordings.